

General Overview

The OpenSearch Load Tester consists of three core modules, each running as its own Spring Boot application:

- **Test Data Generator** - Responsible for creating and indexing synthetic test data in the target OpenSearch instance.
- **Load Generator** - Executes the actual load test by sending parallel queries to the target OpenSearch instance.
It also collects performance metrics during execution and forwards them to the Metrics Reporter.
- **Metrics Reporter** - Collects and exposes all test metrics, making them easily accessible for analysis.
- **UI** - User-friendly configuration and execution of load tests.

All three components, along with the OpenSearch test target, are orchestrated and managed using a single docker-compose setup.

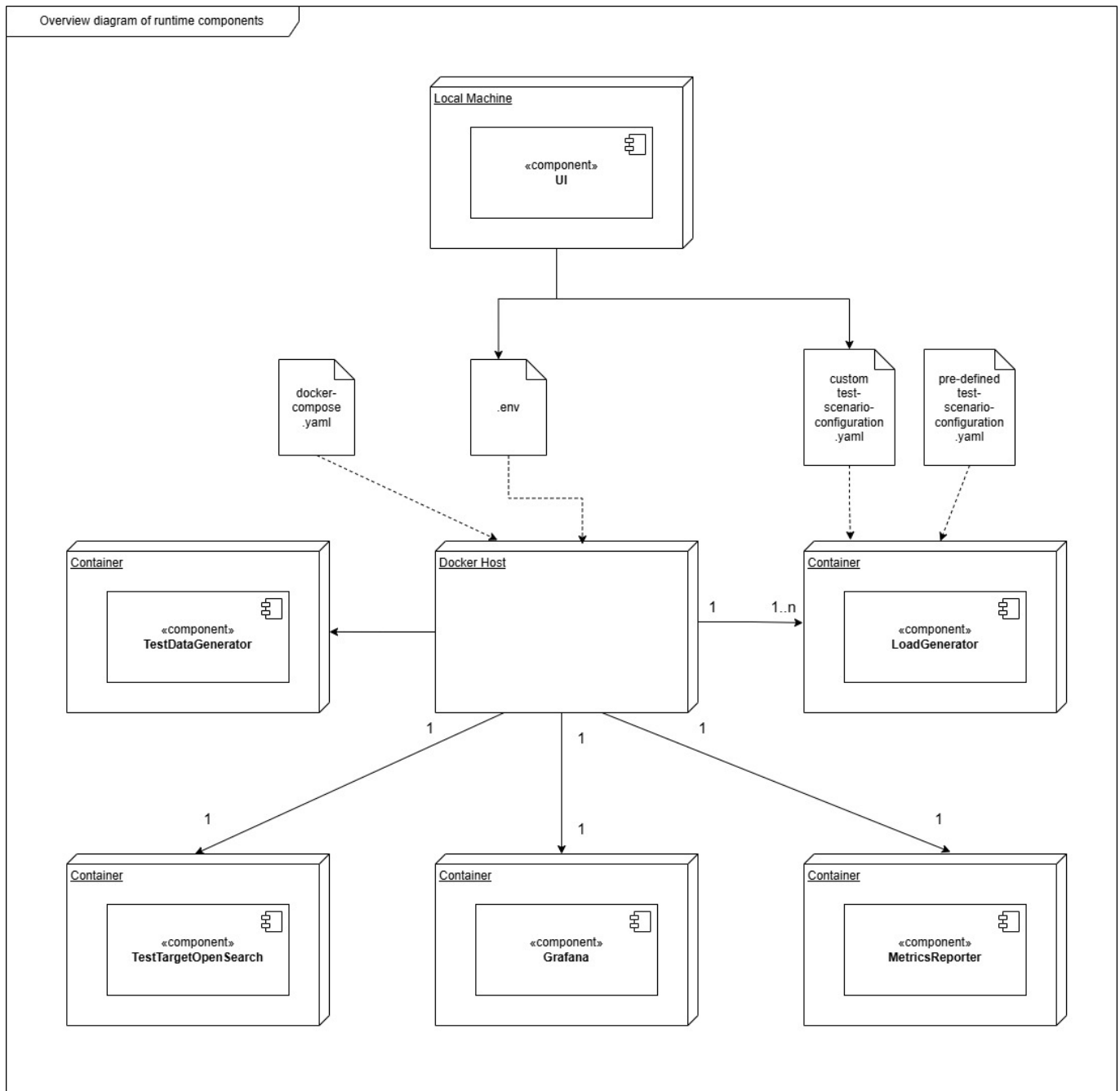


Figure: OpenSearch Load Tester core runtime components architecture

Test Data Generator

The Test Data Generator module is a Spring Boot application responsible for populating a target OpenSearch with synthetic test data. This module generates and indexes configurable test documents into OpenSearch, providing realistic data sets for benchmarking and performance testing of the OpenSearch Load Tester application.

Key Features

- **Multiple Document Types** - Supports different document schemas (**ANO** and **DUO**) with distinct index mappings
- **Flexible Data Generation Modes** - Configurable generation strategies (**dynamic** or **persistent** (Beta))

- **Automatic Initialization** - Executes automatically on application startup via `CommandLineRunner`
- **File Storage Support** - Ability to save and load generated documents locally
- **Big Data Support** - Ability to generate and index large volumes of test data efficiently

Configuration

Behavior is controlled through `DataGenerationProperties` in `application.properties`, allowing customization of the following parameters:

- `count`: Total number of test documents to generate and index
- `batch-size`: Number of documents to generate and index per batch (default: 1000)
- `document-type`: Type of documents to generate (`ANO` or `DUO`)
- `mode`: Data generation mode (`dynamic` or `persistent` (Beta))
- `output-path`: File path for saving and loading generated documents in `persistent` mode (default: `data/testdata.json`)

Note on Persistent Mode: Persistent mode is currently in beta. Persistence to the local file storage is not yet performed in batches. As a result, using a very high `count` may lead to memory issues. It is recommended to keep the count low or use Dynamic mode instead.

Usage

Run as a standalone Spring Boot application or Docker container. The module automatically creates the configured index, generates test data and refreshes the index for immediate visibility.

Architecture

image

Data Generation_INITIALIZER

The **Data Generation_INITIALIZER** is the startup entry point of the Test Data Generator application. It is responsible for triggering and orchestrating the complete test data preparation process when the service starts.

Configuration-driven behavior

The initializer relies on `DataGenerationProperties` to determine:

- `documentType` (e.g. `ANO`, `DUO`)
- `mode` (`DYNAMIC` | `PERSISTENT`)

Based on these properties, it selects the appropriate index definition and initiates the data generation workflow.

Startup workflow

- **Index selection:** Chooses the index model based on `documentType`:
 - `ANO` → `AnoIndex`

- `DUO` → `DuoIndex`

- **Index creation:** Calls `openSearchDao.createIndex(...)` with the index name, settings, and mappings.
- **Data generation & indexing:** Delegates the complete generation and indexing process to `DataGenerationService.generateAndIndexTestData(indexName)`.
- **Index refresh:** Ensures all indexed documents are immediately searchable by calling `openSearchDao.refreshIndex(indexName)`.

Division of responsibilities

- Index lifecycle operations are delegated to **OpenSearch DAO**.
- Data generation and batching logic are delegated to **Data Generation Service**.

The initializer acts as a thin, deterministic startup coordinator.

Data Generation Service

The **Data Generation Service** is the central orchestration and control component of the Test Data Generator. It manages the end-to-end workflow of generating test documents and indexing them into OpenSearch, without creating document content itself or directly interacting with the OpenSearch client. The service interprets `DataGenerationProperties` to control its execution flow.

Workflow orchestration

- **Entry point:** `generateAndIndexTestData(indexName)` logs start and completion and branches based on the configured mode.
- **Dynamic mode:** Documents are generated in batches (`generateData(documentType, batchSize)`) and immediately indexed via `openSearchDao.bulkIndexDocuments(...)`. Progress is logged as `generatedDocs / totalCount`.
- **Persistent mode:** A complete dataset is loaded or generated once via the **Document Generator** and then indexed in batches.

Resilience against OpenSearch backpressure (HTTP 413 / 429)

If `OpenSearchDao` throws an `OpenSearchDataAccessException` caused by a `ResponseException`:

- For **413 (Payload Too Large)** or **429 (Too Many Requests)**, the batch size is dynamically reduced (`batchSize / 2`) and the operation is retried after a short delay.
- If the batch size can no longer be reduced, the exception is escalated.

Division of responsibilities

- Document creation is delegated to the **Document Generator** (`DataGenerator`).
- All OpenSearch communication is delegated to the **OpenSearch DAO** (`OpenSearchDao`).

Document Generator

The **Document Generator** is responsible for creating synthetic test documents. It encapsulates all data generation logic and is fully independent of indexing, batching, or persistence concerns.

Public API (**DataGenerator**)

This interface allows the Data Generation Service to generate data without depending on a concrete implementation.

DynamicDataGenerator

- Generates random documents entirely in memory.
- Dispatches generation based on **DocumentType**:
 - **ANO** → **AnoDocument.random()**
 - **DUO** → **DuoDocument.random()**

PersistentDataGenerator (Beta Version)

- Enables reuse of identical datasets across multiple executions.
- First attempts to load existing data from **FileStorageService**.
- If no data is available, new documents are generated via **DynamicDataGenerator**, persisted, and then returned.

File Storage

File Storage provides the persistence layer for document collections in persistent generation mode. It encapsulates file system access and JSON serialization so that other components remain storage-agnostic.

Save behavior

- Creates required directories and the target file if they do not exist.
- Writes the complete **List<Document>** to disk as JSON.
- I/O errors are propagated as **FileStorageException**.

Load behavior

- If the file does not exist, returns an empty list.
- Otherwise, deserializes the file into **List<targetClass>** using a Jackson **CollectionType**.
- Returns a defensive copy (**new ArrayList<>(...)**) to avoid unintended side effects.

OpenSearch DAO

The **OpenSearch DAO** encapsulates all interactions with OpenSearch and serves as the dedicated data access layer of the Test Data Generator. The DAO provides a clean, high-level abstraction for OpenSearch operations, shielding higher layers from the OpenSearch Java client API and transport details.

- **Index existence check**
 - Uses the OpenSearch Indices API to verify whether an index already exists.
- **Index creation**

- Creates an index only if it does not exist.
- Supports optional index settings and mappings.
- Uses OpenSearch defaults when settings or mappings are omitted.
- **Bulk document indexing**
 - Converts document collections into OpenSearch **BulkOperations**.
 - Executes bulk indexing using the Bulk API.
 - Detects partial failures and aggregates error details.
- **Index refresh**
 - Forces a refresh to make newly indexed documents searchable immediately.

OpenSearch Client

The **OpenSearch Client** component creates and provides the OpenSearch Java client that allows the Test Data Generator to communicate with an OpenSearch cluster. It reads the OpenSearch endpoint from the application configuration, sets up the connection, and exposes the client as a Spring bean so it can be reused by other components, mainly the OpenSearch DAO.

Load Generator

The Load Generator is responsible for executing load tests against the target OpenSearch instance and reporting performance metrics to the Metrics Reporter. It consists of the following core code components:

TestScenarioInitializer

Initializes and starts the load test on application startup:

- optionally runs a warm-up phase
- synchronizes global start across replicas
- invokes the LoadRunner to execute the ScenarioConfig
- reports success or failure to the MetricsReporterClient.

ScenarioConfigLoader.class

Reads test scenario configuration from a YAML file and maps it into a **ScenarioConfig** object.

LoadTestStartSyncClient

- Registers the load generator as READY with the Metrics Reporter
- polls the shared synchronization status until global start is allowed
- waits until the planned start time.

Query Strategy

- Generate parameterize query instances matching test data values
- fill out query template

AbstractQuery.class

Base class that manages query parameters and the query template path

- provides shared helpers
- returns the template path together with resolved parameters for execution.

Available Queries

File	Short description
AnoClientByYearQuery.java	Generates a query that filters Ano clients by a specific year
AnoClientsAggregationQuery.java	Builds an aggregation query over Ano clients (counts/metrics)
AnoDisMaxExpensiveQuery.java	Creates a dis_max query tuned to surface expensive Ano documents
AnoDisMaxQuery.java	Produces a general dis_max query for Ano documents
AnoMultiRegexQuery.java	Generates a query with multiple regex conditions for Ano fields
AnoPayrollRangeQuery.java	Builds a payroll range query for Ano
AnoPayrollTypeLanguageQuery.java	Generates a query filtering by payroll type and language for Ano
AnoPrefixMatchQuery.java	Creates a prefix match query on Ano fields
AnoPrefixRangeQuery.java	Combines prefix and range criteria for Ano
DocNameRegexQuery.java	Produces a regex-based query against document names
DuoBookingByAmountRangeQuery.java	Generates a booking query filtering by amount range for Duo
DuoBookingByClientAndStateQuery.java	Builds a booking query that filters by client and state
DuoBookingByCostcenterAndDateQuery.java	Creates a booking query filtering by cost center and date
DuoClientByCustomerNumberQuery.java	Generates a query retrieving Duo clients by customer number
DuoClientByNameAndStateQuery.java	Builds a query filtering Duo clients by name and state
DuoComplexQuery.java	Produces a composite/complex query combining multiple criteria for Duo data

File	Short description
<code>DuoDateRangeQuery.java</code>	Generates a date-range query for Duo documents (from/to)
<code>DuoInvoiceCategoryQuery.java</code>	Builds an invoice query filtering by category for Duo
<code>DuoInvoiceDisMaxExpensiveQuery.java</code>	Creates a <code>dis_max</code> query targeting expensive Duo invoices
<code>DuoInvoiceDisMaxQuery.java</code>	Produces a general <code>dis_max</code> query for Duo invoices
<code>DuoMultiPrefixSortQuery.java</code>	Generates a query with multiple prefix filters and sorting for Duo
<code>DuoMultiRegexQuery.java</code>	Builds a query using multiple regex filters for Duo fields
<code>DuoStateLocationQuery.java</code>	Creates a query that filters Duo documents by state and location
<code>span/AnoSpanNearQuery.java</code>	Produces a <code>span_near</code> positional query for Ano fields
<code>span/DuoSpanNearQuery.java</code>	Generates a <code>span_near</code> positional query for Duo fields
<code>specialized/AnoMoreLikeThisQuery.java</code>	Builds a <code>more_like_this</code> query for Ano to find similar documents
<code>specialized/DuoMoreLikeThisQuery.java</code>	Creates a <code>more_like_this</code> query for Duo to find similar documents

QueryExecutionTask

Executes a single OpenSearch query from a JSON template

- measures client-side round-trip time
- maps and handles errors to determine a status code
- parses response hit counts and durations

and reports the resulting metrics to the MetricsCollector.

QueryPoolBuilder.class

Constructs a weighted pool of QueryType values from a ScenarioConfig's queryMix, where

- textual entries add a single instance
- object entries with type and percent add the type repeated percent times for weighted selection.

LoadRunner

- Schedules and executes queries from a ScenarioConfig across worker threads
- ensures duration and error handling
- flushes metrics
- performs graceful shutdown and cleanup.

MetricsCollector

- Collects `MetricsDto` objects into a buffer
- when the configurable batch size is reached or on flush sends batches to the `MetricsReporterClient`

MetricsReporterClient

Serializes metrics data and finish signals to JSON.

- POSTs metrics data to `/api/metrics` Metrics Reporter endpoint
 - POSTs finish signals to `/api/finish` Metrics Reporter endpoint
-

Metrics Reporter

Design overview of the Metrics Reporter service that collects load-test metrics, aggregates them, and exports JSON reports.

Purpose and scope

- Accept metrics from one or more load-generator replicas.
- Coordinate synchronized start across all load generators.
- Stream metrics to disk immediately (memory-efficient).
- Aggregate statistics and generate final reports.

Data flow

1. Load generators signal readiness via `POST /api/load-test/ready/{id}`.
2. Once all expected generators are ready, `GET /api/load-test/status` returns `isStartAllowed=true` with a planned start time.
3. Load generators POST metrics batches to `/api/metrics` during the test.
4. Metrics are immediately streamed to NDJSON file and statistics are accumulated in-memory.
5. Each generator calls `POST /api/finish` when done; once all have finished, final reports are generated.
6. The application shuts down gracefully after sending the response.

Core code components

- `MetricsReporterApplication`: Spring Boot entrypoint.
- `ReportController` (`.../controller/ReportController.java`): REST API for metrics submission and finalization (`/api/metrics`, `/api/finish`, `/api/health`).
- `LoadTestStartSyncController` (`.../controller/LoadTestStartSyncController.java`): REST API for synchronized start (`/api/load-test/ready/{id}`, `/api/load-test/status`).
- `ReportService` (`.../service/ReportService.java`): Handles streaming persistence to NDJSON, report finalization, and file I/O.
- `LoadTestStartSyncService` (`.../service/LoadTestStartSyncService.java`): Tracks generator readiness and calculates planned start time.
- `StatsAccumulator` (`.../service/StatsAccumulator.java`): Accumulates min/max/avg statistics in-memory during the test.
- `StatisticsDto` (`.../dto/StatisticsDto.java`): DTO for aggregated statistics output.
- `MetricsDto` (`common-core`): DTO for individual query results with Jakarta Bean Validation.

- **ShutdownAfterResponseInterceptor** (`.../config/ShutdownAfterResponseInterceptor.java`): Initiates graceful shutdown after HTTP response completion.

API contract

Metrics endpoints (`/api`):

- **POST** `/api/metrics`: Receives a list of **MetricsDto** objects. Streams to NDJSON and updates in-memory statistics. Returns HTTP 409 if called after finalization.
- **POST** `/api/finish`: Signals load generator completion. Triggers report generation when all expected generators have finished.
- **GET** `/api/health`: Simple liveness check.

Synchronization endpoints (`/api/load-test`):

- **POST** `/api/load-test/ready/{loadGeneratorId}`: Marks a generator as ready. Idempotent.
- **GET** `/api/load-test/status`: Returns synchronization status (`expectedLoadGenerators`, `readyLoadGenerators`, `isStartAllowed`, `plannedStartTimeMillis`).

Payload schemas (JSON):

- **POST** `/api/metrics` body: array of **MetricsDto**
 - `loadGeneratorId` (string, required, not blank)
 - `queryType` (string, required, not blank)
 - `requestDurationMillis` (number, required, ≥ 0)
 - `queryDurationMillis` (number, optional, ≥ 0)
 - `totalHits` (number, optional, ≥ 0)
 - `httpStatusCode` (number, required, 100-599)
- **POST** `/api/finish` body: **FinishLoadTestDto**
 - `loadGeneratorId` (string, required, not blank)
 - `success` (boolean, required)
 - `errorMessage` (string, optional)

Report formats

- **NDJSON** (`tmp_query_results.ndjson`): Newline-delimited JSON for streaming writes during the test.
- **JSON** (`query_results.json`): Full JSON array of all query results, converted from NDJSON after finalization.
- **Statistics** (`statistics.json`): Aggregated statistics including min/max/avg for request and query durations, total queries, total errors, and load generator instances.

Configuration (`application.properties`)

- `load.generator.replicas`: Expected replica count (default 1).
- `report.output.directory`: Output directory (default `../reports` in metrics-reporter `application.properties`, often overridden in Docker).
- `report.ndjson.filename`: Temporary NDJSON file.
- `report.resultsjson.filename`: Final query results JSON file.
- `report.stats.filename`: Statistics JSON file.
- Logging: `logging.level.com.opensearchloadtester.metricsreporter`.

Error handling and validation

- Uses Jakarta Bean Validation on `MetricsDto` (validates `loadGeneratorId`, `queryType`, `httpStatusCode` range 100-599, non-negative durations).
- Rejects empty payloads or inconsistent `loadGeneratorId` with HTTP 400.
- HTTP status codes ≥ 400 are counted as errors.
- Returns HTTP 409 if metrics are submitted after finalization.

Startup behavior

- On application startup, the reporter initializes the output directory and deletes any existing report files (`tmp_query_results.ndjson`, `query_results.json`, `statistics.json`) before starting a new run.

Shutdown behavior

Exit codes indicate outcome:

- `0`: Successful completion.
- `1`: Internal reporter error.
- `2`: Load generator reported failure.

Extensibility notes

- To add new statistics, extend `StatsAccumulator` and `StatisticsDto`, then update `ReportService.finalizeReports()`.

Common Core Library

The **common** module contains shared, framework-agnostic building blocks used by all OpenSearch Load Tester components (Test Data Generator, Load Generator, Metrics Reporter, and UI).

What belongs here

- **DTOs** used for inter-service communication (REST payloads, events).
- **Small utilities** that enforce consistent formatting or conventions across modules.

Contents

- `com.opensearchloadtester.common.dto`
 - `MetricsDto`: per-request metric event emitted by a load generator.
 - `LoadTestStartSyncStatusDto`: readiness and start coordination state for distributed tests.
 - `FinishLoadTestDto`: end-of-run status message sent by a load generator.
- `com.opensearchloadtester.common.utils`
 - `TimeFormatter`: UTC and ISO date formatting helpers.